

Building Resilient Distributed Systems

Parallel and Distributed Computing Assignment

Moez Umer - BSCS23150

Part 1 — Analysis of the Problems

Problem 1: Synchronization Issue (Lost Update Problem)

The first problem happens when two users try to edit the same document at the same time. In the current system, both users get the same version of the document from the database and make their own changes.

The problem is that the backend does not check whether another user has already updated the document before saving the new data. Because of this, the second update overwrites the first one.

This issue happens during the update request in the API when multiple users send requests together.

The main reasons behind this problem are:

- No concurrency control
- No version checking
- No locking mechanism
- Race conditions between requests

This is called the Lost Update problem because one user's changes get lost.

Problem 2: Coordination Issue (Dropped Webhooks)

The second problem is related to Clerk webhooks. When a user cancels their premium subscription, Clerk sends a webhook event to the backend.

In the current system, if the internet connection or network fails while sending the webhook, the backend never receives the cancellation event.

As a result:

- Clerk marks the user as cancelled
- Backend still keeps the user premium

This means both systems become out of sync.

The main reasons behind this problem are:

- No retry system
- No backup queue
- No idempotency handling
- Dependence on network delivery

The system incorrectly assumes that webhook delivery will always succeed.

Problem 3: Fault Tolerance Issue (LLM API Failure)

The third problem happens because the FastAPI backend makes synchronous requests to an external LLM API.

If the external API becomes slow or stops responding, the backend keeps waiting for a response for a long time.

While waiting, server resources get blocked and other users also get affected. If many requests pile up together, the whole application can become slow or unresponsive.

The external API becomes a single point of failure.

The main reasons behind this problem are:

- Blocking synchronous requests
- No timeout handling
- No fallback response
- No circuit breaker mechanism

This can cause failures to spread across the application.

Part 2 — Proposed Solutions

Solution 1: Optimistic Locking for Synchronization

To solve the lost update problem, Optimistic Locking can be used.

In this approach, every document contains a version number.

When a user updates a document, the backend first checks whether the version sent by the user matches the current version in the database.

If both versions are the same, the update succeeds and the version number increases.

If the versions are different, it means another user already changed the document, so the backend rejects the request with HTTP 409 Conflict.

This prevents old data from overwriting newer data.

UML Sequence Diagram

```
sequenceDiagram
    participant UserA
    participant UserB
    participant FastAPI
    participant Database

    UserA->>FastAPI: GET Document
    FastAPI->>Database: Fetch Document
    Database-->>FastAPI: Version = 1
    FastAPI-->>UserA: Document v1

    UserB->>FastAPI: GET Document
    FastAPI->>Database: Fetch Document
    Database-->>FastAPI: Version = 1
    FastAPI-->>UserB: Document v1

    UserA->>FastAPI: PUT Update (v1)
    FastAPI->>Database: Check Version
    Database-->>FastAPI: Match
    FastAPI->>Database: Save Update
    Database-->>FastAPI: Version = 2
    FastAPI-->>UserA: Success

    UserB->>FastAPI: PUT Update (v1)
    FastAPI->>Database: Check Version
    Database-->>FastAPI: Version Mismatch
    FastAPI-->>UserB: 409 Conflict
```

Solution 2: Reliable Webhook Handling

To solve the webhook problem, the system should use retries, idempotency keys, and dead-letter queues.

If a webhook fails because of a network issue, the system should automatically retry sending it after some time.

Each webhook event should also contain a unique event ID. If the same event is received multiple times, the backend can safely ignore duplicates.

If the webhook keeps failing repeatedly, it can be stored in a dead-letter queue for later checking.

This helps keep both systems synchronized.

Solution 3: Circuit Breaker for Fault Tolerance

To handle failures from the external LLM API, the system should use the Circuit Breaker pattern.

Normally, requests go to the external API.

But if the API keeps failing or becomes too slow, the circuit breaker temporarily stops sending requests to it.

Instead of waiting for a long time, the backend immediately returns a fallback response such as:

```
{  
  "message": "AI service temporarily unavailable"  
}
```

After some time, the system checks whether the external API has recovered.

This prevents the whole application from hanging.

CAP Theorem Trade-Offs

According to the CAP theorem, distributed systems cannot fully guarantee all three properties together:

- Consistency
- Availability
- Partition Tolerance

Since network failures can always happen, partition tolerance is necessary.

The optimistic locking solution focuses more on consistency because conflicting updates are rejected.

The webhook retry system focuses more on availability and eventual consistency because updates may happen later after retries.

The circuit breaker focuses more on availability because the application continues working even if the external API fails.

Conclusion

This assignment shows how distributed systems can face synchronization, coordination, and fault tolerance problems in real-world applications. By using optimistic locking, reliable webhook handling, and circuit breaker patterns, the application becomes more reliable and better able to handle failures.

This assignment demonstrates how distributed systems fail under synchronization, coordination, and fault tolerance challenges. By implementing optimistic locking, reliable webhook processing, and circuit breaker patterns, the application becomes more resilient, consistent, and fault tolerant under real-world distributed conditions.